

Actin quantum automata: Communication and computation in molecular networks



S. Siccardi^{b,*}, A. Adamatzky^a

^a Unconventional Computing Center, University of the West of England, Bristol, UK

^b Department of Physics, University of Milan, via Celoria 16, 20133 Milano, Italy

ARTICLE INFO

Article history:

Received 12 January 2015

Accepted 20 January 2015

Available online 2 February 2015

Keywords:

Cellular automata

Quantum automata

Molecular networks

Logical operations

Adders

ABSTRACT

Actin is filament-forming protein forming a communication and information processing cytoskeletal network of eukaryotic cells. Actin filaments play a key role in developing synaptic structure, memory and learning of animals and humans; many psychiatric and neurological disorders are due to disfunction in the assembly of actin fibres. This is why it is important to develop abstractions of the information processing on the actin filaments. We model actin filaments as two chains of one-dimensional quantum automata arrays to describe hypothetical signalling events propagating along the chains. We study in detail several functions of automaton state transitions and compute examples of evolution to illustrate behaviour of the functions and a role of the superposition of the initial states. We uncover and analyse localisations, or particles, propagating along the actin chains. We demonstrate that logical gates can be realised in the result of the collisions. Using collisions between the travelling particles we implement binary adder.

© 2015 Elsevier Ltd. All rights reserved.

1. Introduction

Two key components of a cytoskeleton—actin double helix (micro)filaments and tubule microtubule protofilaments. They are responsible for keeping a cell shape and implementing the cell's motility but also they play a role of the cell's nervous system: together with extracellular matrix [1] the intracellular networks of proteins process information and implement learning [2–10]. By modulating dendritic ion channel activity actin filaments determine rule of neural information processing and facilitate computational abilities of dendritic trees via facilitation of ionic condensation and ion cloud propagation [11]. Psychiatric and neurological disorders are caused by disfunctions in actin assembly or the actin association with other proteins and intracellular components [12–17].

Computational models tubule microtubules have been developed in 1990s and used to demonstrate that computation could be implemented in tubulin protofilaments by classical and quantum means [3,18–20]. Less attention was paid to actin double helix filaments, despite importance of the actin in learning and information pre-processing as might be hinted by pre-dominant presence of actin networks in synapses [21–24]. In our previous paper [25] we proposed a model of actin filaments as two chains of one-dimensional binary-state semi-totalistic automaton arrays to hypothetical signalling events. We analyse the complete rule space of actin automata using integral characteristics of space–time configurations generated by these rules and compute state transition rules that support travelling and mobile localisations. We found that some properties of actin automata rules may be predicted using Shannon entropy, activity and incoherence of excitation between the polymer chains. We also show that it is possible to infer whether a given rule supports travelling or stationary localisations by looking at ratios of excited neighbours that are essential for generation of the localisations. In the

* Corresponding author.

E-mail address: ssiccardi@2sas.it (S. Siccardi).

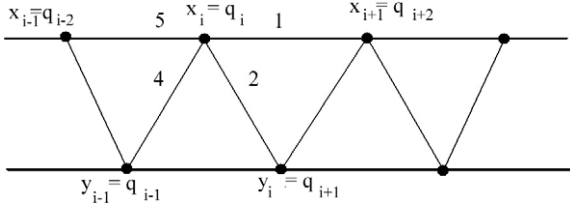


Fig. 1. A scheme of quantum actin automata.

present work we continue the line of investigation, using a quantum cellular automaton (QCA).

The actin double helix is an attractive candidate system for describing how data is processed and transmitted within a cell. Exact mechanisms for many of these processes have yet to be fully understood. We propose that a computation is implemented when localisations (defects, conformational changes, ionic clouds, solitons) representing data collide with each other and undergo state changes. Assuming that values of the computer variables are encoded in states of the localisations we come to the concept of collision-based computing [26–29]. The potential or ionic waves [30] or quantum protein transitions [20,31] might play roles of signal passing through the proteins and branches of the cytoskeletal network, colliding, changing their velocities or states, and thus performing Boolean logical operations [26].

Our aim is to show the computational power of the quantum automaton, without describing a detailed physical or chemical mechanism that could realise it in actin. Our basic hypothesis is that the ‘hardware’ is made of two layers of cells, whose internal state is influenced by four adjacent cells (Fig. 1). Although circuitual models of actin have been proposed, where electrical pulses could be identified with signals [30], in the present work we will consider signals without making any specific hypotheses on their nature. We suppose that they can travel left to right and vice versa along each layer of the filament and between the two layers. Accordingly, we factor the state of each cell in five substates, one representing the specific excitation of the cell, the others the four pulses that can arrive from the neighbouring cells. The same model could be employed to deal with other systems, like e.g. kinks in zig-zag configurations of ion traps (see e.g. [32]).

2. Quantum cellular automata

In quantum cellular automata (QCA) cell states are represented by qubits or cubits, instead of classical bits. Several formal definitions have been proposed, see e.g. [33–35], and also methods to translate classical automata into quantum ones can be found in [36,37]. However, it is often difficult to check if a specific candidate system fulfils all the requirements to be a QCA; also the simulation of the evolution of a QCA is in general a complex task.

We start our discussion with two classical definitions of one dimensional QCA, 1QCA, due to Watrous [38,39,33].

Definition 1. A 1QCA $\mathcal{A} = \langle Q, \lambda, N, \delta \rangle$ is determined by a finite set Q of states, a *quiescent state* λ , a neighbourhood

$N = \{n_1, \dots, n_r\} \subseteq \mathbb{Z}$, with $n_1 < n_2 < \dots < n_r$ and a local transition function

$$\delta : Q^{r+1} \rightarrow C_{[0,1]} \quad (1)$$

satisfying the following three conditions:

(1) local probability: for any $(q_1, \dots, q_r) \in Q^r$,

$$\sum_{q \in Q} |\delta(q_1, \dots, q_r, q)|^2 = 1 \quad (2)$$

(2) stability of the quiescent state: if $q \in Q$, then

$$\delta(\lambda, \dots, \lambda, q) = 1 \quad \text{if } q = \lambda, 0 \text{ otherwise.} \quad (3)$$

To state the third condition we have to introduce the automaton’s mapping.

A configuration $c : \mathbb{Z} \rightarrow Q$ is a mapping such that $c(i) \neq \lambda$ only for finitely many i . Let $C(\mathcal{A})$ denote the set of all configurations. Computation of $\mathcal{A}$ is done in the space $H_{\mathcal{A}} = l_2(C(\mathcal{A}))$ with the basis $\{|c\rangle \mid c \in C(\mathcal{A})\}$.

In one step, $\mathcal{A}$ transfers from one basis state $|c_1\rangle$ to another $|c_2\rangle$ with the amplitude

$$\alpha(c_1, c_2) = \prod_{i \in \mathbb{Z}} \delta(c_1(i + n_1), \dots, c_1(i + n_r), c_2(i)). \quad (4)$$

A state in $H_{\mathcal{A}}$, in general, has the form

$$|\phi\rangle = \sum_{c \in C(\mathcal{A})} \alpha_c |c\rangle \quad (5)$$

with normalised α_c . The evolution operator $E_{\mathcal{A}}$ of $\mathcal{A}$ maps any state $|\phi\rangle$ into $|\psi\rangle = E_{\mathcal{A}}|\phi\rangle$ such that

$$|\psi\rangle = E_{\mathcal{A}}|\phi\rangle = \sum_{c \in C(\mathcal{A})} \beta_c |c\rangle \quad (6)$$

where

$$\beta_c = \sum_{c' \in C(\mathcal{A})} \alpha_{c'} \alpha(c', c). \quad (7)$$

Now we can state the third condition:

(3) unitarity: the mapping $E_{\mathcal{A}}$ must be unitary.

In general, to decide whether the unitarity condition is satisfied is a nontrivial problem [40–42]. Partitioned quantum one-dimensional cellular automata, that are well suited to represent our cells and their states, are easier to deal with.

Definition 2. A partitioned quantum one-dimensional cellular automaton (P1QCA) is a 1QCA $\mathcal{A} = \langle Q, \lambda, N, \delta \rangle$ that satisfies the following conditions:

(1p) The set of states Q is the cartesian product $Q = Q_1 \times \dots \times Q_r$ of $r = |N|$ nonempty sets.

(2p) The local transition function δ is the composition of two functions:

$$\delta_c : Q^r \rightarrow Q \quad \text{a classical mapping} \quad (8)$$

$$\delta_q : Q \rightarrow C^Q \quad \text{a quantum mapping} \quad (9)$$

where

$$\begin{aligned} \delta_c((q_{1,1}, \dots, q_{1,r}), \dots, (q_{r,1}, \dots, q_{r,r})) \\ = (q_{r,1}, q_{r-1,2}, \dots, q_{1,r}). \end{aligned} \quad (10)$$

The function δ_q defines a 1QCA with operator

$$U_{\mathcal{A}_q}(q_2, q_1) = [\delta_q(q_1)](q_2). \quad (11)$$

$U_{\mathcal{A}_q}$ is also called the local transition matrix of $\mathcal{A}$. It can be shown that the evolution of a P1QCA is unitary if and only if this operator is unitary.

3. Application to actin-like structures

Our next step is to translate the actin automaton defined in [25] to a quantum automaton based on the P1QCA definition.

The original actin automaton is composed of two layers of cells, labelled x_k and y_k . In our model we use only one type of cells, q_i , identifying $q_{2k+1} = x_k$ and $q_{2k} = y_k$ like in Fig. 1.

With this convention, the neighbour of any q_i is

$(q_{i-2}, q_{i-1}, q_i, q_{i+1}, q_{i+2})$, that corresponds both to the neighbour of x_k and y_k cells in the original notation.

Moreover, the state of the generic cell q_i is a vector $(q_{i1}, q_{i2}, q_{i3}, q_{i4}, q_{i5})$ and the first step of the evolution includes building of the intermediate state

$$\tilde{q}_i = (q_{i+2,1}, q_{i+1,2}, q_{i,3}, q_{i-1,4}, q_{i-2,5}). \quad (12)$$

In the classical actin automaton [25], the node/cell transition rules have the following form

$$\begin{aligned} &((a_{00}, a_{01}, a_{02}, a_{03}, a_{04}), (a_{10}, a_{11}, a_{12}, a_{13}, a_{14})) \\ &= (X_0, X_1) \end{aligned} \quad (13)$$

with $a_{ik} \in \{0, 1\}$ and X_i 5-elements arrays. To compute the state of cell q_i at time $t + 1$, the sum

$$S = q_{i-2} + q_{i-1} + q_{i+1} + q_{i+2} \quad (14)$$

is computed at time t , and the new state of q_i takes the value $a_{q_i(t), S}$ where $q_i(t)$ is the state of q_i at time t .

In the quantum case, we must specify the matrix (of dimension 32) $U(q_1, q_2) = [u_{a_1 a_2 a_3 a_4 a_5, b_1 b_2 b_3 b_4 b_5}]$ whose elements give the probability that a state $\tilde{q} = (a_1 a_2 a_3 a_4 a_5)$ evolves in another state $q = (b_1 b_2 b_3 b_4 b_5)$.

For the generic element of U we have:

$$u_{a_1 a_2 a_3 a_4 a_5, b_1 b_2 b_3 b_4 b_5} = 0 \quad \text{if } b_3 \neq X_{a_3, S} \quad (15)$$

and

$$\begin{aligned} u_{a_1 a_2 a_3 a_4 a_5, b_1 b_2 b_3 b_4 b_5} &= 1 \quad \text{if } b_3 = X_{a_3, S} \\ \text{and } b_{i \neq 3} &= B_i(S) \end{aligned} \quad (16)$$

where $B_i(S)$ is an array or a rule, possibly dependent on S , that must be specified. In other terms, in the P1QCA case we have to specify also how the four 'extra' states evolve. We will show that several choices are possible (Eqs. (17), (18) or (19)), that makes this kind of automata quite flexible and suitable to model different situations. In other words, when using quantum automata we have three choices to make: (1) the specific definition of the automaton, e.g. a partitioned quantum automaton; (2) the transition matrix type; (3) the rule for the evolution, like in the classical case.

Note that:

(1) the value $\lambda = (0, 0, 0, 0, 0)$ must be stable, so the rules $(1, x, y, z, w)$, (any) are excluded;

(2) we have to check if the matrix U is unitary for specific rules;

(3) the above automaton can be very similar to a classical one: we can use a mechanism that generates superposition of the states or does not; in the latter case, we can start with cells in superposition or without. If we start with classical – not superposed – states, the evolution is classical;

(4) we used the following definition to manage the states transmission:

$$\begin{aligned} u_{a_1 a_2 a_3 a_4 a_5, b_1 b_2 b_3 b_4 b_5} &= 1 \quad \text{if } b_3 = X_{a_3, S} \\ \text{and } b_{i \neq 3} &= a_i \end{aligned} \quad (17)$$

that is, the four "extra" substates of a node/cell are copied from the neighbours of the node/cell. For some examples, we used also a variant of this matrix that swaps states 1 and 5, 2 and 4 if state 3 is active.

We also evaluated the definition:

$$\begin{aligned} u_{a_1 a_2 a_3 a_4 a_5, b_1 b_2 b_3 b_4 b_5} &= \chi / 4 \quad \text{if } b_3 = X_{a_3, S} \\ \text{and any } b_{i \neq 3} & \end{aligned} \quad (18)$$

where $\chi = \pm 1$ is computed in order to obtain a unitary 16×16 matrix.

Moreover we tested the definition:

$$\begin{aligned} u_{a_1 a_2 a_3 a_4 a_5, b_1 b_2 b_3 b_4 b_5} &= \chi / 2 \quad \text{if } b_3 = X_{a_3, S} \\ \text{and } \sum_{b_{i \neq 3}} &= S = 1 \text{ or } 3 \end{aligned} \quad (19)$$

where $\chi = \pm 1$ is computed in order to obtain a unitary matrix; and u is computed according to (17) for $S \neq 1$ and $S \neq 3$.

4. Computation

4.1. Unitarity

We performed unitarity check, as explained in the above discussion, for the 512 rules considered in the classical actin automata (excluding rules 1xxxx because of quiescent states stability). We used definition (17), where a cell inherits the substates of its neighbours, definitions (18) and (19) with superposition of all the substates but the central one, giving the same sum as the substates at the previous step, when the sum value is 1 or 3. In all the cases we found 16 unitary matrices, for rules:

f1 = (00000);	f2 = (11111) = (0, 31) identity
f1 = (00001);	f2 = (11110) = (1, 30)
f1 = (00010);	f2 = (11101) = (2, 29)
f1 = (00011);	f2 = (11100) = (3, 28)
f1 = (00100);	f2 = (11011) = (4, 27)
f1 = (00101);	f2 = (11010) = (5, 26)
f1 = (00110);	f2 = (11001) = (6, 25)
f1 = (00111);	f2 = (11000) = (7, 24)
f1 = (01000);	f2 = (10111) = (8, 23)
f1 = (01001);	f2 = (10110) = (9, 22)
f1 = (01010);	f2 = (10101) = (10, 21)
f1 = (01011);	f2 = (10100) = (11, 20)
f1 = (01100);	f2 = (10011) = (12, 19)

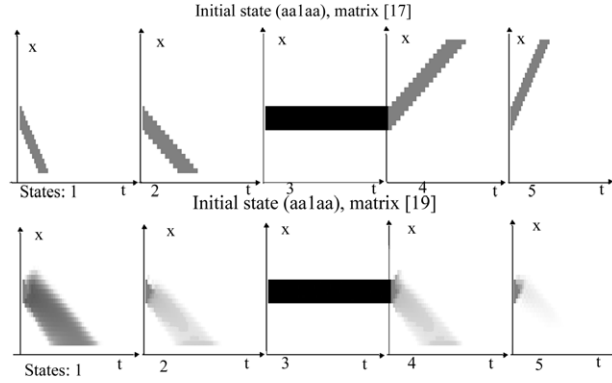


Fig. 2. Rule (0, 31), initial state is (aa1aa) as explained in the text, upper panel matrix (17), lower panel matrix (19).

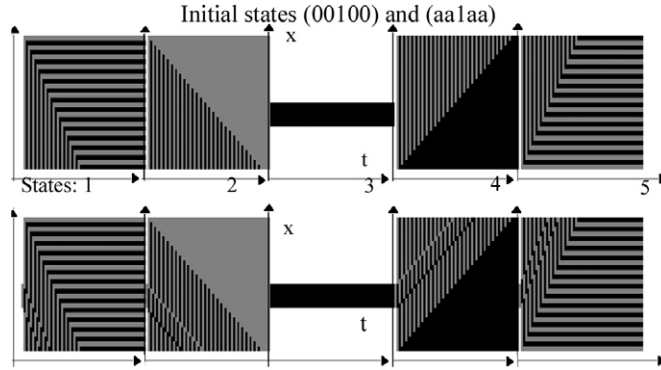


Fig. 3. Rule (0, 31), matrix (18), upper panel is initial state (00100), lower panel is initial state (aa1aa) as explained in the text.

$$\begin{aligned}
 f1 &= (01101); & f2 &= (10010) = (13, 18) \\
 f1 &= (01110); & f2 &= (10001) = (14, 17) \\
 f1 &= (01111); & f2 &= (10000) = (15, 16).
 \end{aligned}$$

It is worth noting that for all the rules, conditions that excite a quiescent cell will deactivate an excited one.

4.2. Evolution

We computed evolutions for the unitary matrices found in the above computations.

In the first runs, we used seeds like those shown in our previous paper [25], 5 x_i and 5 y_i cells, all the others being in the quiescent state. The automata had a total of 60 cells and the automaton was evolved for 60 steps.

As our aim at this stage was a qualitative comparison of the classical and quantum models, we tried to obtain comparable results, so we prepared pictures in terms of x and y (that is q_{2n+1} and q_{2n}) cells.

Figs. 2–3 show the evolution of the 5 substates of the x cells; time is the horizontal axis and x the vertical axis. We recall that the states are actually qubits, that is they are a linear combination of the two basis elements $|0\rangle$ and $|1\rangle$. Accordingly, the pictures represent the module of the coefficient of $|1\rangle$ in grey, black meaning 1 and white 0.

Of course, it is expected that the identity rule (0, 31) will leave the cells in their initial condition, and it is indeed the case starting with 5 x_i and 5 y_i cells in the state (00100).

It is worth to note that, if we start with cells in a superposition state, e.g. $1/4 \sum (a_j a_k 1 a_i a_l)$, with all the combinations of $a = 0$ and $a = 1$, we get an evolution for the 'lateral' substates, even with the rule (0, 31), as shown in Fig. 2, upper panel matrix (17), lower panel matrix (19).

On the other hand, matrix (18) causes an evolution of the lateral substates both with (00100) and superposed initial states, see Fig. 3. Actually these pictures are quite similar, showing that the superposition generated by the matrix overcomes the influence of initial conditions.

It is interesting to compare these results with the evolutions of seeds obtained with the classical automaton, e.g. Fig. 4 taken from [25] (Fig. 19, panels b and l). The graphs have been rotated in order to have the same axes as the quantum automaton pictured for ease of comparison.

We did not aim at a systematic analysis of seeds, but looked for seeds that are particles (gliders, localisations) moving to the right or to the left and that could be used as building blocks for some logical functions. Examples of evolutions starting with one seed generating a particle moving to the right and another seed generating a particle move in to the left are shown in Fig. 5. In this and in the subsequent pictures, we make no distinctions between x_i and y_i , and the pictures show the evolution of all the cells together, that will be labelled generically x .

Seeds are just two sets of three adjacent cells each, at positions (6, 7, 8) and (12, 13, 14). Cell 7 is initialised at 00010, cell 8 and 12 at 00100, cell 13 at 01000. Cells 6 and 14 carry in their rightmost and leftmost states respectively

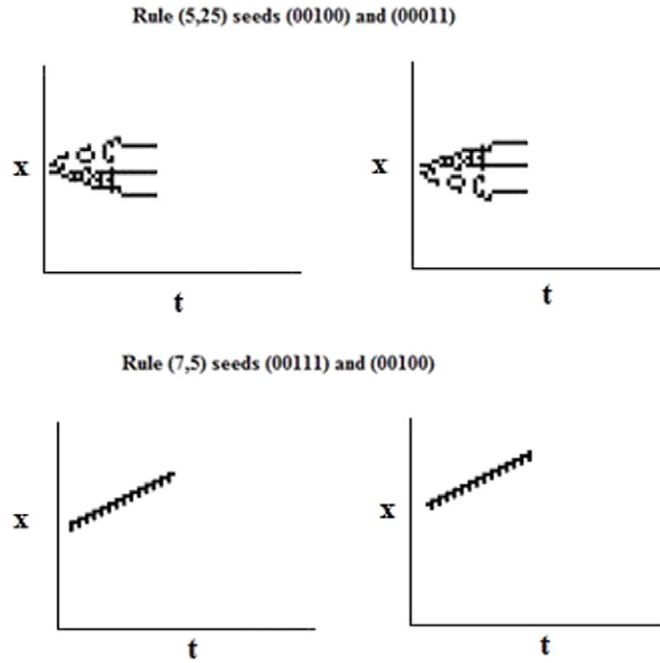


Fig. 4. Evolution of the classical automaton of [25], upper panel rule (5, 25) seeds (00100) and (00011), lower panel rule (7, 5) seeds (00111) and (00100).

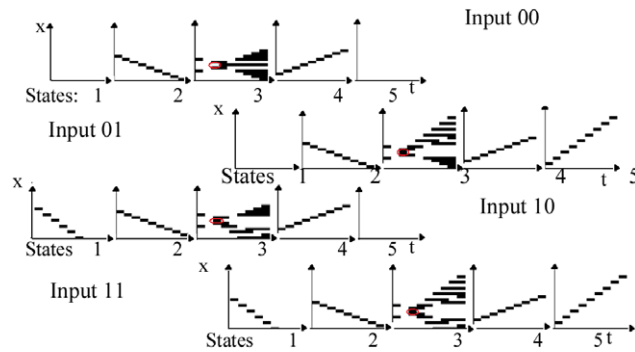


Fig. 5. Rule (14, 17), matrix (17) of two cells at positions 6 and 14 in initial right and left states 00, 01, 10, 11. The ‘output’ of the logical OR gate is the red circles. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

the bits that we want to use in an operation. In the four runs we have the four combinations 00, 01, 10 and 11.

When the seeds meet at the third step, the resulting central state of cell 10 (circled in red in the picture) is 1 unless both cells 8 and 12 are initialised at 0. This means that this automaton is acting like a classical gate OR: if it could be ‘prepared’ in the initial states containing the two seeds at specified positions, and if it could be ‘read’ at the middle position after the specified number of steps, it could be used as a building block for a classical computer.

The same automaton – when initialised in a superposition of states – propagates superposition. For instance one could initialise cells 7 and 13 at a superposition of excited and idle state, e.g. $1/\sqrt{2} \cdot 00010 + 1/\sqrt{2} \cdot 00000$ and $1/\sqrt{2} \cdot 01000 + 1/\sqrt{2} \cdot 00000$ respectively and let cells 8 and 12 start at 00100 like in the previous case. Cells 6 and 14 that carry the qubits used in the operation are initialised at $1/2 \cdot 00000 + \sqrt{3}/2 \cdot 00001$ and $1/2 \cdot 00000 + \sqrt{3}/2 \cdot 10000$. What

one obtains is that the output state (state 3 of cell 10 at step 3) is 0 if both inputs are 0, it is $1/2|0\rangle + \sqrt{3}/2|1\rangle$ if one of the inputs is in the active state $1/2 \cdot 00000 + \sqrt{3}/2 \cdot 00001$, and it is $1/4|0\rangle + \sqrt{15}/4|1\rangle$ if both input cells are active.

So we see that although the automaton is perfectly working with quantum states, the translation of classical to quantum logical operations is not completely straightforward.

5. Catalogue of particles and their collisions for matrix (17)

We now turn to a systematic study of the behaviour of matrix (17) with classical input, that is with particles that are not superposed states. Here we use term ‘particle’ which is analogue of glider in one-dimensional, i.e. a compact patten of non-quiescent states being translated along the automaton array during repeated application of

the state transition rules. The particle could be an abstraction of conformational defects, kinks, ionic clouds, solitons or impulses.

We recall that matrix (17) means that the substates other than the central one are copied from the neighbouring cells, and the central one is transformed accordingly to one of the 16 rules listed in Section 1.

5.1. Moves

We have 32 possible initial states of a single particle, and we now study how each of these states evolves depending on the rule that is used. We omit rule (0, 31) that is the identity and sometimes cell $0 = \langle 00000 \rangle$ that is in the resting state. The particle is placed in position n at time t and we look at what happens in the next few steps, until a stable behaviour is reached.

For all the rules, (00100) is a fixed point.

The rules are split into two groups with regard to the type of moves they generate. Rules of the first group have the activating part of the rule between 0 and 7: this means that they require at least two lateral active substates in a particle to activate the central substate, and this cannot happen starting with a single cell.

On the other hand, rules of the second group have the activating part of the rule between 8 and 15, so they can activate the central substate of a particle even if just one of the lateral substates is active.

For rules in the first group, a particle having one active bit only moves one or two cells for each time step. For rules in the second group, particles having one active bit are transformed in two-bit particles of type (00101), (00110), (01100) or (10100), and produce (00100) particle behind at each step. The evolution of cells having more than one active bits is split into the evolution of the individual bits.

Some results are graphically presented in Figs. 6–8, eight cases for each picture, time on the horizontal axis goes from left to right and cells positions are on the vertical axis. Particles are drawn as boxes that can contain up to five dots, each dot representing an active substate. The observer is thought to look at evolution from behind the vertical axis, so that “right” cells are lower and “left” cells are upper. For the sake of simplicity the cells are represented in a single line, but it must be noted that they are arranged in two lines as in actin (e.g. odd cells in one filament and even cells in the other).

5.2. Collisions between two particles

We now consider the collisions of two particles, with all the possible combinations of initial states, starting at two adjacent positions, or with one, two or three cells beneath them. Apart from identity (0, 31), we find four groups of rules:

- (1, 30) = (2, 29) = (3, 28);
- (4, 27) = (5, 26) = (6, 25) = (7, 24);
- (8, 23) = (9, 22) = (10, 21) = (11, 20);
- (12, 19) = (13, 18) = (14, 17) = (15, 16),

the rules of each group having the same behaviour for two particles collisions.

For rules (1, 30) = (2, 29) = (3, 28) collisions have no effects, that is two particles evolve in the same manner that each of them would have done if it were the only active one, whatever their initial state and relative position. This is due to the fact that these rules require at least three active lateral substates to activate the central one, and this cannot happen starting with two active cells only.

For rules (4, 27) = (5, 26) = (6, 25) = (7, 24), instead, we have that the central bit can be activated in the “collided” particle. Which exact collided particle and which exact substate in the initially activated particles are involved depend on the initial relative position of the particles. For instance in the case of two particles that are initially adjacent, the next particle to the left (resp. right) of the initial couple is activated if both substate 2 of the left particle and 1 of the right particle (resp. 5 of the left particle and 4 of the right particle) are active. This is a logical AND gate. Examples are shown in Fig. 9. When there is a resting cell between the two initially activated, a similar evolution takes place, but the “target” cell is one more place to the left (see Fig. 10). When we increase the gap between the initially activated cells, the target moves further to the left or to the right.

Moreover, increasing the initial gap, also other substates of the initial particle (e.g. 1 of the left particle and 4 of the right particle) can activate one of the cells in the gap. The total combinations are 448 for adjacent colliding particles, that is: 256 combinations having an active substate 2 of the left particle and 1 of the right particle, 256 having an active substate 5 of the left particle and 4 of the right particle, minus those having active 2 and 5 substates of the left particle and 1 and 4 of the right particle that are 64 (because they should not be counted twice). The combinations are 592 for colliding particles having a single cell between them, 624 for colliding particles separated by two cells and 640 for particles separated by three cells. In total we have 2304 collisions.

For rules (8, 23) = (9, 22) = (10, 21) = (11, 20), if the particles are adjacent, we have that the central bit is activated in the “collided” left (resp. right) parcel if substates 2 of the particle or 1 of the right particle (resp. 5 of the left particle or 4 of the right particle) are active, but not both. This is a logical gate XOR. It must be noted that in this case the consequence of a collision may be that the central state that would be activated in the move of a single particle, is not activated during collision of two particles. For example, compare the second and third moves in Fig. 7 with the first collision in Fig. 11.

Also for these rules, when the initial gap between colliding particles is increased other substates of the particles can interact. The total combinations are 828 for adjacent particles, 907 for particles having a single cell between them, 848 for particles separated by two cells and 896 for particles separated by three cells. In total we have 3479 collisions.

For rules (12, 19) = (13, 18) = (14, 17) = (15, 16), in case of adjacent particles, we have that the central bit is activated in the “collided” left (resp. right) particle if substates 2 of the left particle or 1 of the right particle (resp. 5 of the left particle or 4 of the right particle) are active. This is a logical gate OR.

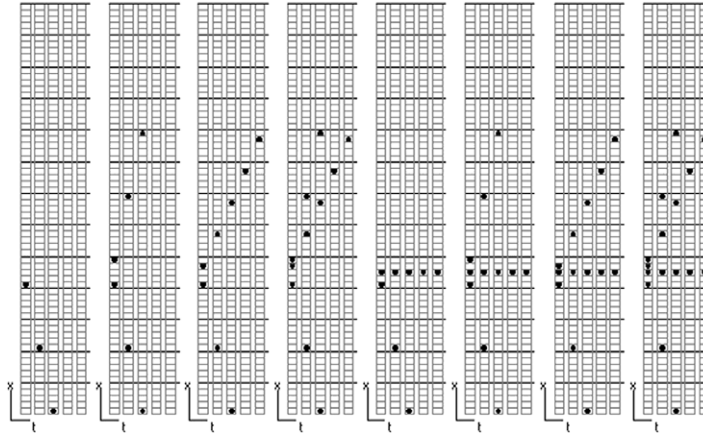


Fig. 6. Moves of a single particle; rules of the first group, initial conditions 16–23.

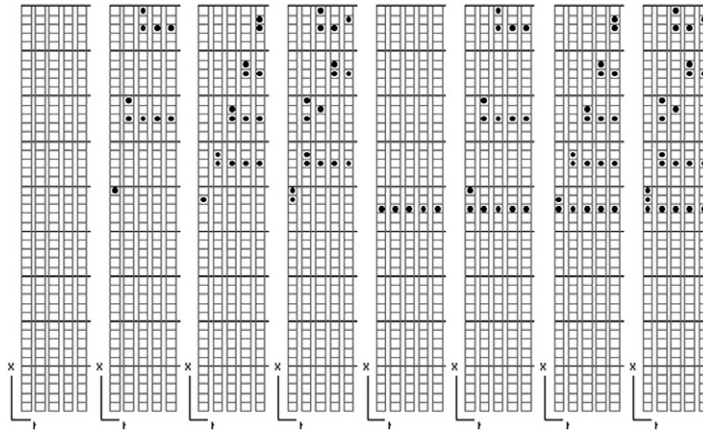


Fig. 7. Moves of a single particle; rules of the second group, initial conditions 0–7.

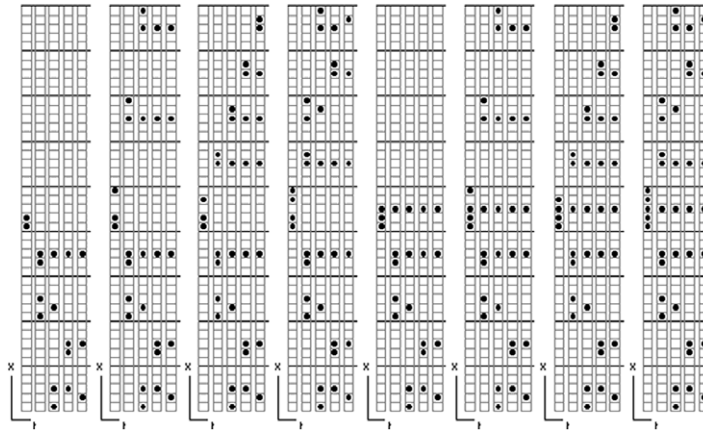


Fig. 8. Moves of a single particle; rules of the second group, initial conditions 24–31.

The total combinations are 828 for adjacent particles, 903 for particles having a single cell between them, 832 for particles separated by two cells and 911 for particles separated by three cells. In total we have 3474 collisions.

Finally, we show that with these rules it is possible to build simple gates. In Fig. 12 there is an OR gate using rule

(12, 19). The input bits are substate 1 of cell 6 and substate 2 of cell 7, and the output is substate 3 of cell 8.

In the same manner it is possible to build AND and XOR gates, however there are two restrictions: (1) each rule group can build only one type of gate; (2) it is not possible to send the output of a gate (substate 3) to the input of

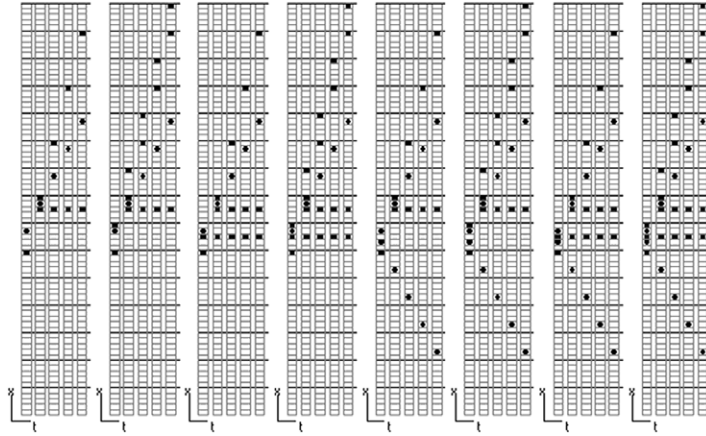


Fig. 9. Collision of adjacent particles; rules $(4, 27) = (5, 26) = (6, 25) = (7, 24)$, initial conditions as in the first column of each group.

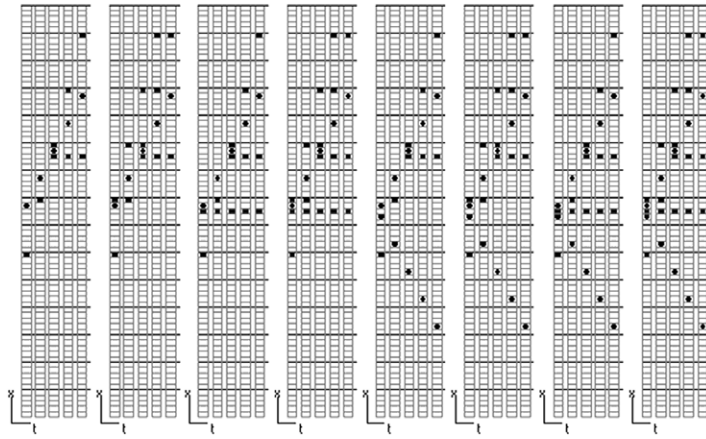


Fig. 10. Collision of particles with a one cell gap; rules $(4, 27) = (5, 26) = (6, 25) = (7, 24)$, initial conditions as in the first column of each group.

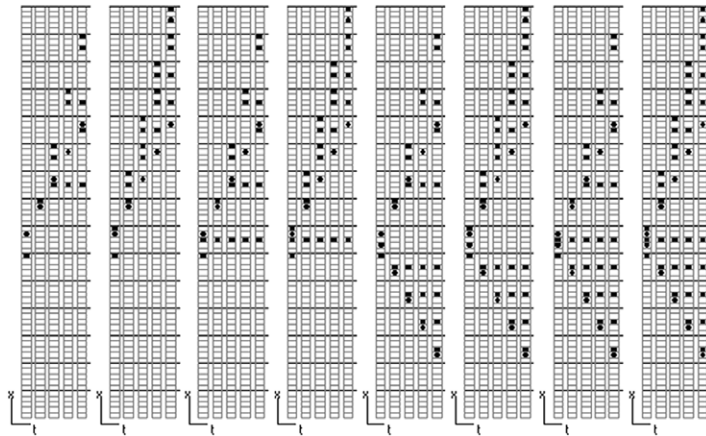


Fig. 11. Collision of adjacent particles; rules $(8, 23) = (9, 22) = (10, 21) = (11, 20)$, initial conditions as in the first column of each group.

another gate (that requires to move the information to another substate).

We will see that the first problem can be solved using collisions of more than two particles, while the second one requires a slightly different matrix.

5.3. Collisions of five particles

We now study the general case of evolution of given consecutive particles, say particles $n, n + 1, n + 2, n + 3, n + 4$.

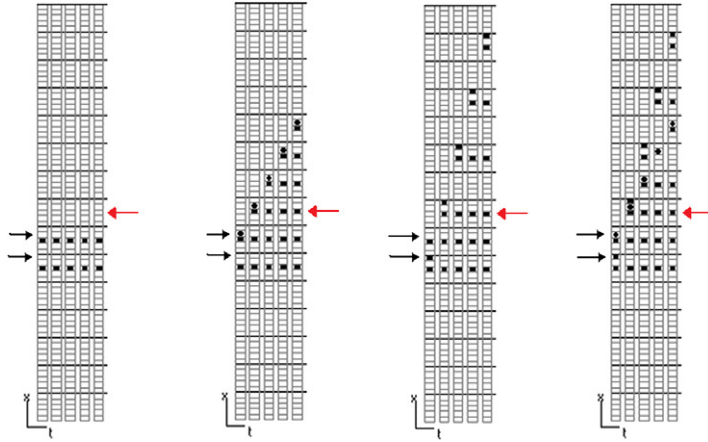


Fig. 12. A simple OR gate using rule (12, 19), inputs in cells 6 and 7 (black arrows), outputs are in cell 8 (red arrow). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

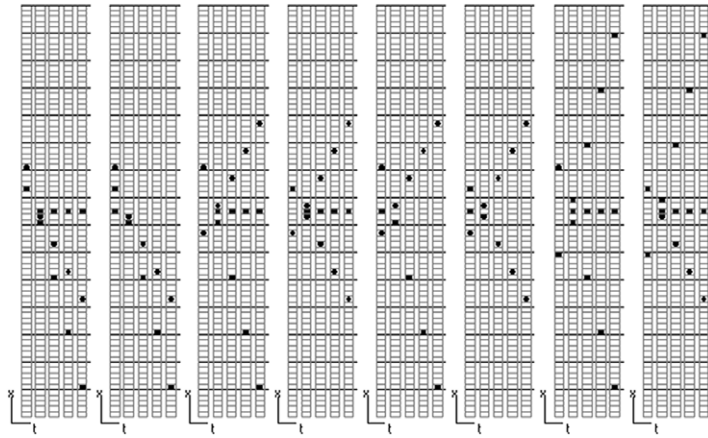


Fig. 13. Collisions for rule (4, 27)—part 1.

As the number of possible initial states is huge (32^5), we limit ourselves to the initial states that influence the evolution of the central cell $n + 2$, that is one substate of each of the 5 cells, namely substate 5 (the rightmost) of cell n , substate 4 of cell $n + 1$, substate 3 of cell $n + 2$, substate 2 of cell $n + 3$ and substate 1 (the leftmost) of cell $n + 4$.

In this way we have 32 initial conditions for each of the 15 rules. The runs show that in this case each rule has some specific behaviour, different from all the other rules, so we do not have groups like in the 2 particles case. Moreover, every rule produces collisions at least for some initial conditions, that is there are at least some initial conditions that evolve in a different way than they would do if the particles were evolving independently.

In Figs. 13–14 we represent for example some evolutions for rule (4, 27) that are not equivalent to individual cells.

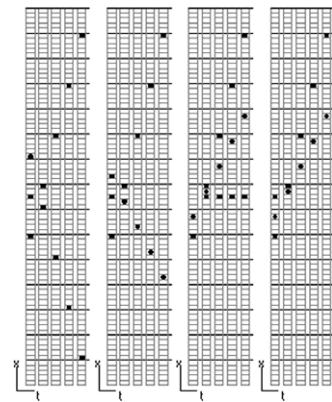


Fig. 14. Collisions for rule (4, 27)—part 2.

6. Gates

We now discuss all the logical gates we can build in the evolution of given consecutive cells described in the previous section.

We recall that, considering two input qubits, we have 16 logical gates as in Table 1. With five cells, each carrying an active substate, that is a qubit, we can choose the input qubits in 10 different ways, and for each choice we can choose each of the five qubits as the output. Moreover,

Table 1

Logical gates definition, columns contain the output value, given the input values of the headings.

(0, 0)	(0, 1)	(1, 0)	(1, 1)	gate
0	0	0	0	constant 0
0	0	0	1	$x \wedge y$
0	0	1	0	$x \wedge \neg y$
0	0	1	1	$x \vee y$
0	1	0	0	$\neg x \wedge y$
0	1	0	1	$y \vee x$
0	1	1	0	$x \oplus y$
0	1	1	1	$x \vee y$
1	0	0	0	$\neg x \wedge \neg y$
1	0	0	1	$x \oplus \neg y$
1	0	1	0	$\neg y \vee x$
1	0	1	1	$x \vee \neg y$
1	1	0	0	$\neg x \vee y$
1	1	0	1	$\neg x \vee y$
1	1	1	0	$\neg(x \wedge y)$
1	1	1	1	constant 1

Table 2

Logical gates depending on the choice of 2 input qubits, 1 output qubit and the initial state of the qubits other than the 2 input ones. Rule (4, 27), time step 1, ancillary states 000–011.

inp1	inp2	out	000	001	010	011
1	2	3	$x \wedge y$	$x \oplus y$	$x \oplus y$	$\neg x \wedge \neg y$
1	3	3	$y \vee x$	$x \oplus y$	$x \oplus y$	$x \oplus \neg y$
1	4	3	$x \wedge y$	$x \oplus y$	$\neg(x \wedge y)$	$x \oplus \neg y$
1	5	3	$x \wedge y$	$x \oplus y$	$\neg(x \wedge y)$	$x \oplus \neg y$
2	3	3	$y \vee x$	$x \oplus y$	$x \oplus y$	$x \oplus \neg y$
2	4	3	$x \wedge y$	$x \oplus y$	$\neg(x \wedge y)$	$x \oplus \neg y$
2	5	3	$x \wedge y$	$x \oplus y$	$\neg(x \wedge y)$	$x \oplus \neg y$
3	4	3	$x \vee y$	$x \oplus y$	$x \oplus y$	$x \oplus \neg y$
3	5	3	$x \vee y$	$x \oplus y$	$x \oplus y$	$x \oplus \neg y$
4	5	3	$x \wedge y$	$\neg(x \wedge y)$	$x \oplus y$	$x \oplus \neg y$

Table 3

Logical gates depending on the choice of two input qubits, one output qubit and the initial state of the qubits other than the two input ones. Rule (4, 27), time step 1, ancillary states 100–111.

inp1	inp2	out	100	101	110	111
1	2	3	$\neg(x \wedge y)$	$x \oplus \neg y$	$x \oplus \neg y$	$x \vee y$
1	3	3	$x \oplus y$	$x \oplus \neg y$	$x \oplus \neg y$	$y \vee x$
1	4	3	$x \oplus y$	$\neg x \wedge \neg y$	$x \oplus \neg y$	$x \vee y$
1	5	3	$x \oplus y$	$\neg x \wedge \neg y$	$x \oplus \neg y$	$x \vee y$
2	3	3	$x \oplus y$	$x \oplus \neg y$	$x \oplus \neg y$	$y \vee x$
2	4	3	$x \oplus y$	$\neg x \wedge \neg y$	$x \oplus \neg y$	$x \vee y$
2	5	3	$x \oplus y$	$\neg x \wedge \neg y$	$x \oplus \neg y$	$x \vee y$
3	4	3	$x \oplus y$	$x \oplus \neg y$	$x \oplus \neg y$	$x \vee y$
3	5	3	$x \oplus y$	$x \oplus \neg y$	$x \oplus \neg y$	$x \vee y$
4	5	3	$x \oplus y$	$x \oplus \neg y$	$\neg x \wedge \neg y$	$x \vee y$

for each choice of the input qubits we can choose in eight different ways the initial state of the other three ‘ancillary’ qubits.

So, for each rule, we have $10 \times 5 \times 8 = 400$ possible gates, and we run the evolution for the values (0, 0), (0, 1), (1, 0), (1, 1) to check which logical gates it is possible to obtain. Moreover, we considered three time steps, in order to check if the results are stable, or if we could have different gates, just waiting one or two more time steps.

As expected, the first rules like (1, 30) give less interesting results than others, that can excite states starting with a lower number of cells.

Results for rule (4, 27) are reported in Tables 2 and 3; the first time step of the collision is reported, similar results have been found for the second and third steps later. As it is clear that by construction one gets interesting results only choosing substate 3 for output, only these combinations have been considered in the tables.

7. Adders

Using the gates shown in Tables 2 and 3 for rule (4, 27), we will now build a simple two-bit half-adder.

We use substates 5 and 4 as input bits for an AND gate, to obtain the “carrier” or more significant digit of the sum; then we use an auxiliary bit in substate 2 of a cell on the right to meet substate 5 in another AND gate. The result, in substate 3, goes downwards and meets the input substate 4 in XOR gate (using another auxiliary bit that comes down from the right), and gives the less significant digit of the sum.

The half adder is shown in Fig. 15.

At this point, we have shown that the automaton is powerful enough to realise several gates; however it has a problem, as the output (substate 3) of a gate cannot be directly sent as input to another gate. As we anticipated, to solve this problem a slightly different matrix must be considered.

Actually, it is easy to change the matrix (17) to another one that makes active substates 1 and 2 to be swapped to substates 5 and 4, when they meet in a cell that has an active substate 3. In this way it is possible, e.g. to get a half adder whose output state for the carrier is substate 2 of cell 3 at step 6 (Fig. 16).

In such a way it is possible, in principle, to send the output of a gate to another one in an arbitrarily long chain. However, as states go on moving without the possibility of being turned off, it is very difficult to program even slightly more complex circuits, like e.g. a full adder, because particles interact in a lot of unwanted ways.

To get something useable it is mandatory to implement separate automata, each one performing a specific task. In such a way we can program for instance a full adder. We limit ourselves, for the sake of brevity, to examples $0 + 1 + 0 = 1$, $1 + 0 + 1 = 10$ and $1 + 1 + 1 = 11$ shown in Figs. 17–19.

8. Discussion

We have described a simple partitioned quantum automaton that could be implemented using a two layer physical system. Cells should be equipped with a principal internal state (that is state 3 of our automaton) and other four substates reflecting the signals arriving from the four nearest cells. The principal state is supposed to be influenced by the other substates in a way that can be described by a unitary matrix. We have shown that several quite different matrices can be considered, some generating superposition (if the system is intrinsically a quantum system), others not having this characteristic.

We then studied in detail the behaviour of the automaton in a classical setting, that is using a matrix that does not generate superposition and classical input states, and showed that even in this simple framework it is possible to build a rich set of gates and a full adder. We used the rule

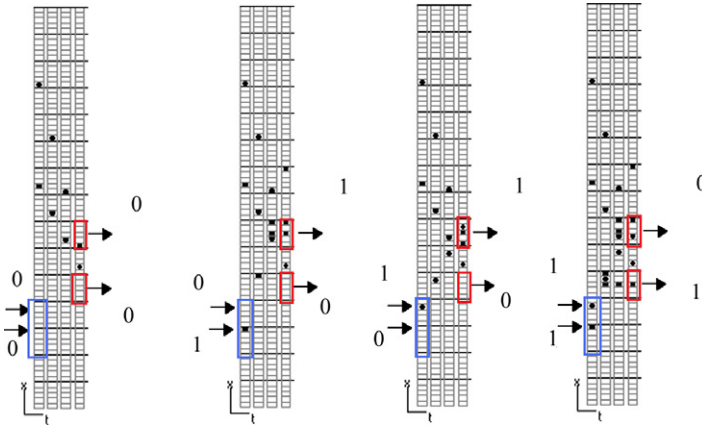


Fig. 15. A two-bit half-adder with gates built using rule (4, 27). Blue squares: input cells, red square: output cells. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

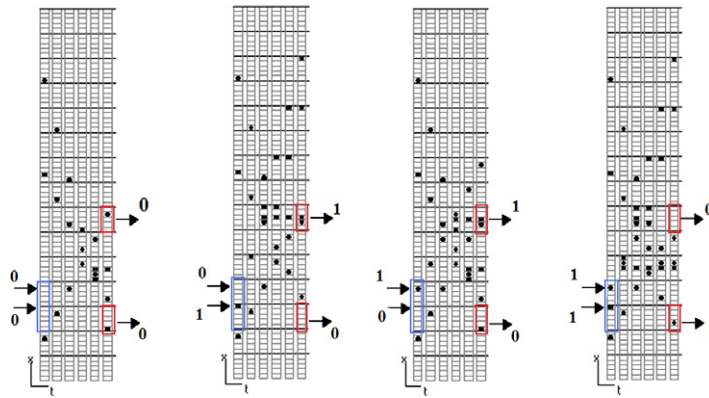


Fig. 16. A two-bit half-adder with gates built using rule (4, 27). Blue squares: input cells, red square: output cells, substate 3 for the sum and substate 2 for the carrier. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

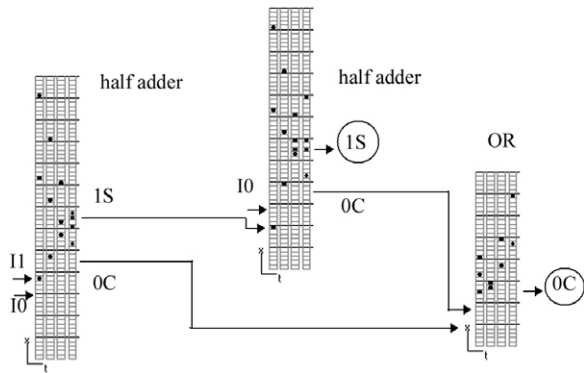


Fig. 17. A three-bit full-adder with gates built using rule (4, 27): $0 + 1 + 0 = 1$. The final sum and carrier are encircled.

(4, 27) for our computations, that is the rule that activates a resting cell when exactly two neighbours are active, and deactivates (makes resting) an active cell in the same circumstance. We computed tables, like Tables 2 and 3, for all the rules, and it turns out that other rules have the same power, e.g. (6, 25), (7, 24), (8, 23) and others.

The most important point at this stage is related to implementation. The next step will be to check if some of the

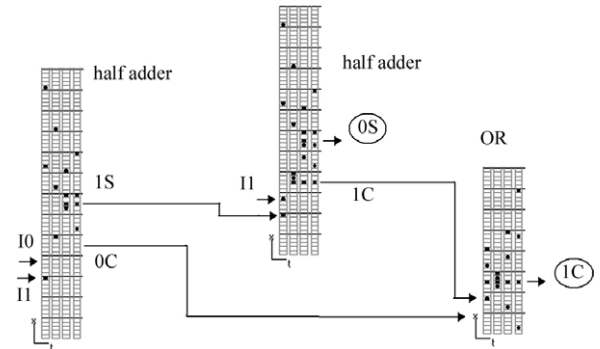


Fig. 18. A three-bit full-adder with gates built using rule (4, 27): $1 + 0 + 1 = 10$. The final sum and carrier are encircled.

circuit models of actin that have been proposed can be interpreted in terms of pulses that activate and deactivate the states of the cells in a way that corresponds to our hypotheses. In other words, it will be necessary to compute a mechanism through which pulses can have a kind of constructive interference to activate a state, and a destructive interference to deactivate it. The natural time step of the system will probably play an important role. When considering real systems, it may be that our simple automaton

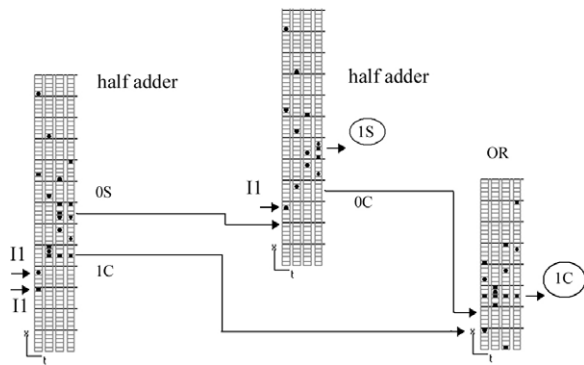


Fig. 19. A three-bit full-adder with gates built using rule (4, 27): $1 + 1 + 1 = 11$. The final sum and carrier are encircled.

will not be sufficient and that we must turn to a more complex model; we believe however that our partitioned quantum automaton and its behaviour will be useful to guide the search.

Of course, there can be even more effects in a physical system that must be taken into account, if one wants to make a real model, that in principle could be built. For instance one must take into account errors, that can arise when the signals travel or are composed. Moreover, the interactions might fade after a while (something that might have both positive and negative effects on the computations), and different sections of the system could follow different rules.

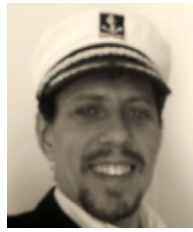
As a last point, if one wants to make a real computer of this system, it is necessary to find an efficient way to prepare the input states and to read the output states and routes for storing information have to be defined.

References

- [1] C.H. Damsky, Z. Werb, Signal transduction by integrin receptors for extracellular matrix: cooperative processing of extracellular information, *Curr. Opin. Cell Biol.* 4 (5) (1992) 772–781.
- [2] S. Hameroff, Coherence in the cytoskeleton: implications for biological information processing, in: *Biological Coherence and Response to External Stimuli*, Springer, 1988, pp. 242–265.
- [3] S. Rasmussen, H. Karampurwala, R. Vaidyanath, K.S. Jensen, S. Hameroff, Computational connectionism within neurons: A model of cytoskeletal automata subserving neural networks, *Physica D* 42 (1) (1990) 428–449.
- [4] B. Ludin, A. Matus, The neuronal cytoskeleton and its role in axonal and dendritic plasticity, *Hippocampus* 3 (S1) (1993) 61–71.
- [5] M. Conrad, Cross-scale information processing in evolution, development and intelligence, *BioSystems* 38 (2) (1996) 97–109.
- [6] J. Tuszynski, J. Brown, P. Hawrylak, Dielectric polarization, electrical conduction, information processing and quantum computation in microtubules. Are they plausible? *Philos. Trans. R. Soc. Lond. Ser. A Math. Phys. Eng. Sci.* (1998) 1897–1925.
- [7] A. Priel, J.A. Tuszynski, H.F. Cantiello, The dendritic cytoskeleton as a computational device: an hypothesis, in: *The Emerging Physics of Consciousness*, Springer, 2006, pp. 293–325.
- [8] D. Debanne, Information processing in the axon, *Nature Rev. Neurosci.* 5 (4) (2004) 304–316.
- [9] A. Priel, J.A. Tuszynski, N.J. Woolf, Neural cytoskeleton capabilities for learning and memory, *J. Biol. Phys.* 36 (1) (2010) 3–21.
- [10] L. Jaeken, A new list of functions of the cytoskeleton, *IUBMB life* 59 (3) (2007) 127–133.
- [11] A. Priel, J.A. Tuszynski, H.F. Cantiello, Electrodynamics signaling by the dendritic cytoskeleton: toward an intracellular information processing model, *Electromag. Biol. Med.* 24 (3) (2005) 221–231.
- [12] A. Van Woerkom, The major hallucinogens and the central cytoskeleton: an association beyond coincidence? towards sub-cellular mechanisms in schizophrenia, *Med. hypotheses* 31 (1) (1990) 7–15.
- [13] J.C. Fiala, J. Spacek, K.M. Harris, Dendritic spine pathology: cause or consequence of neurological disorders? *Brain Res. Rev.* 39 (1) (2002) 29–54.
- [14] A.M. Persico, T. Bourgeron, Searching for ways out of the autism maze: genetic, epigenetic and environmental clues, *Trends Neurosci.* 29 (7) (2006) 349–358.
- [15] Y. Sekino, N. Kojima, T. Shirao, Role of actin cytoskeleton in dendritic spine morphogenesis, *Neurochem. Int.* 51 (2) (2007) 92–104.
- [16] M. van Spronsen, C.C. Hoogenraad, Synapse pathology in psychiatric and neurologic disease, *Curr. Neurol. Neurosci. Rep.* 10 (3) (2010) 207–214.
- [17] N. Kojima, T. Shirao, Synaptic dysfunction and disruption of post-synaptic drebrin–actin complex: a study of neurological disorders accompanied by cognitive deficits, *Neurosci. Res.* 58 (1) (2007) 1–5.
- [18] S.R. Hameroff, J.E. Dayhoff, R. Lahoz-Beltra, A.V. Samsonovich, S. Rasmussen, Models for molecular computation: conformational automata in the cytoskeleton, *Computer* 25 (11) (1992) 30–39.
- [19] P. Janmey, The cytoskeleton and cell signalling: component localization and mechanical coupling, *Physiol. Rev.* 78 (1998) 763–781.
- [20] S. Hameroff, Quantum computation in brain microtubules? the Penrose–Hameroff ‘orch or’ model of consciousness, *Philos. Trans. R. Soc. Lond. Ser. A* 356 (1998) 1869–1896.
- [21] E. Fiková, R.J. Delay, Cytoplasmic actin in neuronal processes as a possible mediator of synaptic plasticity, *The Journal of Cell Biology* 95 (1) (1982) 345–350.
- [22] C.-H. Kim, J.E. Lisman, A role of actin filament in synaptic transmission and long-term potentiation, *J. Neurosci.* 19 (11) (1999) 4314–4324.
- [23] C. Dillon, Y. Goda, The actin cytoskeleton: integrating form and function at the synapse, *Annu. Rev. Neurosci.* 28 (2005) 25–55.
- [24] L.A. Cingolani, Y. Goda, Actin in action: the interplay between the actin cytoskeleton and synaptic efficacy, *Nature Rev. Neurosci.* 9 (5) (2008) 344–356.
- [25] A. Adamatzky, R. Mayne, Actin automata: Phenomenology and localizations, *arxiv* 1408.3676.
- [26] A. Adamatzky, Collision-based computing in biopolymers and their automata models, *Internat. J. Modern Phys. C* 11 (2000) 1321–1346.
- [27] A. Adamatzky, Topics in reaction–diffusion computers, *J. Comput. Theoret. Nanosci.* 8 (2011) 295–303.
- [28] R. Toth, C. Stone, B. de Lacy Costello, A. Adamatzky, L. Bull, Simple collision-based chemical logic gates with adaptive computing, *International Journal of Nanotechnology and Molecular Computation (IJNMC)* 1 (2009) 1–16.
- [29] L. Zhang, A. Adamatzky, Collision-based implementation of a two-bit adder in excitable cellular automaton, *Chaos Solitons Fractals* 41 (2009) 1191–1200.
- [30] J. Tuszynski, S. Portet, J. Dixon, C. Luxford, H. Cantiello, Ionic wave propagation along actin filaments, *Biophys. J.* 86 (2004) 1890–1903.
- [31] S.R. Hameroff, The brain is both neurocomputer and quantum computer, *Cognitive Sci.* 31 (6) (2007) 1035–1045.
- [32] H. Landa, S. Marcovitch, A. Retzker, M.B. Plenio, B. Reznik, Quantum coherence of discrete kink solitons in ion traps, *Phys. Rev. Lett.* 104 (2010) 043004.
- [33] J. Gruska, *Quantum Computing*, McGraw-Hill, 1999.
- [34] P. Delgado, *Quantum cellular automata: theory and applications* (Ph.D. thesis), University of Waterloo.
- [35] B. Aoun, M. Tarifi, Introduction to quantum cellular automata, *arxiv quant-ph* 0401123.
- [36] S. Inokuchi, Y. Mizoguchi, Generalized partitioned quantum cellular automata and quantization of classical ca, *Int. J. Unconventional Comput.* 1 (2005) 149–160.
- [37] B. Schumacher, R.F. Werner, Reversible quantum cellular automata, *arxiv quant-ph* 0405174.
- [38] J. Watrous, On one-dimensional quantum cellular automata, in: *Foundations of Computer Science, 1995. Proceedings, 36th Annual Symposium on, IEEE, 1995*, pp. 528–537.
- [39] A. Kondacs, J. Watrous, On the power of quantum finite state automata, in: *Foundations of Computer Science, 1997. Proceedings, 38th Annual Symposium on, IEEE, 1997*, pp. 66–75.
- [40] J. Gruska, Descriptive complexity issues in quantum computing, *J. Autom. Lang. Comb.* 5 (3) (2000) 191–218.
- [41] C. Durr, H. Lê Thanh, M. Santha, A decision procedure for well-formed linear quantum cellular automata, *arXiv preprint cs/9906024*.
- [42] S. Aaronson, J. Watrous, Closed timelike curves make quantum and classical computing equivalent, *Proc. R. Soc. Ser. A Math. Phys. Eng. Sci.* 465 (2102) (2009) 631–647.



Stefano Siccardi got his Ph.D. degree in Physics, from Milan University, Italy, for his work on quantum information methods for entanglement computation: the case of the helium atom. His research interests are in pragmatic information and formal representations of uncertainty, entanglement computation, exotic states in the dynamical Casimir effect, analysis of biomedical information and data-mining.



Andrew Adamatzky is Professor in the Department of Computer Science and Director of the Unconventional Computing Centre, University of the West of England, Bristol, UK. He does research in reaction–diffusion computing, cellular automata, physarum computing, massive parallel computation, applied mathematics, collective intelligence and robotics, bionics, computational psychology, non-linear science, novel hardware, and future and emergent computation.